

Nama : Frederico Steven Kwok

NPM : 242310037

Kelas : TI-24-PA1

Mata Kuliah : Lab. Pemrograman Perangkat Bergerak

Tugas : Pertemuan 5 – Tugas dan Latihan

Github : <https://github.com/RicoSteven120206/PPB-Praktikum-242310037.git>

Pada tugas praktikum ini, mahasiswa diminta untuk melanjutkan pengembangan fitur pencarian buku yang telah dibuat pada sesi praktikum sebelumnya dengan menambahkan pengolahan data yang lebih dinamis dan efisien. Fokus utama tugas ini adalah penerapan Hooks pada React Native, khususnya `useState`, `useEffect`, dan `useMemo`, dalam menangani perhitungan data dan pengelolaan tampilan berbasis kondisi.

Latihan 1: Menghitung Total Buku Yang Tersedia

Latihan pertama adalah menghitung total data buku yang tersedia maupun yang telah difilter melalui fitur pencarian. Perhitungan ini harus dikelola menggunakan state dan diperbarui secara otomatis ketika terjadi perubahan data, sehingga jumlah buku yang ditampilkan selalu sinkron dengan kondisi aplikasi. Dalam contoh dibawah ini, tulisan Text yang sebelumnya adalah "See All" menjadi "Total x item".

⇒ Berikut Source Code untuk menampilkan pada Search Page ini ketika melakukan pencarian, dan muncul misalkan 3 item atau yang didapat, maka “See All” menjadi “Total x Item”

Source Code /components/BookCollection.jsx

```
import { Ionicons } from "@expo/vector-icons";
import AntDesign from "@expo/vector-icons/AntDesign";
import { useRouter } from "expo-router"; // 1. Tambahkan import useRouter disini
import { Image, Text, TouchableOpacity, View } from "react-native";
import { color_list, styles } from "../styles/StyleApps";

export default function BookCollection({ books = [], isSearching = false }) {
  const totalItems = books.length;
  const sortedBooks = [...books].sort((a, b) => (b.id || 0) - (a.id || 0));
```

```

return (
  <View style={styles.container_book_collections}>
    <View style={styles.h_container}>
      <Text style={styles.container_book_collections_title}>
        Book Collection
      </Text>
      <Text style={{ color: color_list.green }}>
        {isSearching
          ? `Total ${totalItems} item${totalItems > 1 ? "s" : ""}`
          : "See All"}
      </Text>
    </View>

    {totalItems === 0
      ? (
        <View style={styles.not_found}>
          <Text>No Record Found</Text>
        </View>
      )
      : (
        <BookList books={sortedBooks} />
      )
    }

  </View>
);
}

const BookList = ({ books }) => {
  const router = useRouter();

  return (

```

```

<View style={styles.book_grid}>
  {books.map((book, index) => (
    <TouchableOpacity
      key={index}
      style={[styles.book_card, styles.shadow]}
      activeOpacity={0.7}
      onPress={() => router.push(`/books/${book.id}`)}
    >
      <BookItemImg book={book} />
      <BookItemContent book={book} />
    </TouchableOpacity>
  ))}
</View>
);
};

const BookItemImg = ({ book }) => {
  return (
    <View style={{ position: "relative" }}>
      <Image
        source={book.img}
        style={styles.book_card_img}
        resizeMode="cover"
      />
      {!book.is_free && (
        <View style={[styles.circle_premium_small, styles.shadow]}>
          <AntDesign name="crown" size={18} color="black" />
        </View>
      )}
    </View>
  );
};

```

```

const BookItemContent = ({ book }) => {
  return (
    <View style={{ padding: 10 }}>
      <Text
        style={styles.book_card_title}
        numberOfLines={2}
        ellipsizeMode="tail"
      >
        {book.title}
      </Text>
      <Text style={styles.book_card_author} numberOfLines={1}>
        {book.author}
      </Text>
      <View style={styles.book_card_footer}>
        <View style={{ flexDirection: "row", alignItems: "center" }}>
          <AntDesign name="star" size={14} color={color_list.orange} />
          <Text style={styles.book_card_rating}>{book.rating}</Text>
        </View>
        {book.views} && (
          <View style={{ flexDirection: "row", alignItems: "center" }}>
            <Icons name="eye-outline" size={14} color="gray" />
            <Text style={styles.book_card_views}>{book.views}</Text>
          </View>
        )
      </View>
    </View>
  );
};

```

Source Code /components/SearchPage.jsx

```

import { useMemo, useState } from "react";
import { ScrollView, StatusBar, Text, View } from "react-native";
import { SafeAreaView } from "react-native-safe-area-context";
import ListBook from "../../constants/list_book";

```

```

import BookCollection from "../praktikum-4/components/BookCollections";
import SearchBar from "../praktikum-4/components/SearchBar";
import { color_list, styles } from "../styles/StyleApps";

const SearchPage = () => {
  const [search, setSearch] = useState("");
  const [book, setBook] = useState(ListBook);

  const BookDataMemori = useMemo(() => {
    const query = search.trim().toLowerCase();
    if (!query) return book;

    return book.filter((item) => {
      return Object.keys(item).some((key) => {
        try {
          const value = item[key];
          return (
            value != null &&
            String(value).toLowerCase().includes(query)
          );
        } catch (error) {
          console.error(`Error processing key "${key}":`, error);
          return false;
        }
      })
    })
  }, [search, book]);

  const isSearching = search.trim().length > 0;

  return (
    <SafeAreaView style={styles.container}>
      <StatusBar barStyle={"dark-content"} />

```

```

<SearchBar value={search} setValue={setSearch} />
<ScrollView
  showsVerticalScrollIndicator={false}
  style={{ padding: 10 }}
>
  <View style={{ flex: 1 }}>
    <BookCollection books={BookDataMemori} isSearching={isSearching}
  />
  </View>
  <View style={{ justify-content: "flex-end" }}>
    <Text style={{ color: color_list.green, text-align: "center" }}>
      &copy; 2026 Frederico Steven Kwok
    </Text>
  </View>
</ScrollView>
</SafeAreaView>
);
};

export default SearchPage;

```

Latihan 2: Membuat View Untuk Menampilkan Data Tidak Ditemukan

Setiap membangun sebuah aplikasi selalu ada kondisi jika data yang dicari itu tidak ditemukan, pada latihan ke dua ini, buatlah sebuah view bertuliskan "No record found", jika data yang dicari tidak ada.

⇒ Berikut Source Code untuk menampilkan 'No Record Found' Jika terjadi pada pencarian tidak menemukan record item.

```

{totalItems === 0
  ? (
    <View style={styles.not_found}>
      <Text>No Record Found</Text>
    </View>
  )
  : (
    <BookList books={sortedBooks} />
  )
}

```

Latihan 3: Membuat View Paging

Mengimplementasikan fitur paging (pagination) pada komponen daftar buku. Paging harus memiliki limit data per halaman yang dapat diatur, serta menampilkan maksimal lima nomor halaman dalam satu waktu. Apabila jumlah data buku melebihi 100 item, maka sistem paging harus secara otomatis menampilkan lima nomor halaman terakhir sebagai representasi halaman akhir.

⇒ Berikut Source Code untuk Fitur Pagination pada BookCollection dan Pagination

Source Code /components/BookCollection.jsx

```
import { Ionicons } from "@expo/vector-icons";
import AntDesign from "@expo/vector-icons/AntDesign";
import { useRouter } from "expo-router";
import { useEffect, useState } from "react";
import { Image, Text, TouchableOpacity, View } from "react-native";
import { color_list, styles } from "../../styles/StyleApps";
import Pagination from "../../praktikum-5/Pagination";

const LARGE_DATASET_THRESHOLD = 100;

export default function BookCollection({
  books = [],
  isSearching = false,
  itemsPerPage = 4,
}) {
  const totalItems = books.length;
  const sortedBooks = [...books].sort((a, b) => (b.id || 0) - (a.id || 0));
  const totalPages = Math.max(1, Math.ceil(totalItems / itemsPerPage));

  const [currentPage, setCurrentPage] = useState(1);

  // Reset / atur ulang halaman setiap kali data buku berubah (mis. hasil search berubah)
  useEffect(() => {
```

```

    if (totalItems > LARGE_DATASET_THRESHOLD) {
        // 3. Jika data > 100 item, langsung tampilkan 5 nomor halaman terakhir
        setCurrentPage(totalPages);
    } else {
        setCurrentPage(1);
    }
}, [totalItems, totalPages]);

const paginatedBooks = sortedBooks.slice(
    (currentPage - 1) * itemsPerPage,
    currentPage * itemsPerPage
);

return (
    <View style={styles.container_book_collections}>
        <View style={styles.h_container}>
            <Text style={styles.container_book_collections_title}>
                Book Collection
            </Text>
            <Text style={{ color: color_list.green }}>
                {isSearching
                    ? `Total ${totalItems} item${totalItems > 1 ? "s" : ""}`
                    : "See All"}
            </Text>
        </View>

        {totalItems === 0 ? (
            <View style={styles.not_found}>
                <Text>No Record Found</Text>
            </View>
        ) : (
            <
                <BookList books={paginatedBooks} />
            >

```

```

        <Pagination
          currentPage={currentPage}
          totalPages={totalPages}
          onPageChange={setCurrentPage}
        />
      </>
    )}
  </View>
);
}

const BookList = ({ books }) => {
  const router = useRouter();

  return (
    <View style={styles.book_grid}>
      {books.map((book, index) => (
        <TouchableOpacity
          key={index}
          style={[styles.book_card, styles.shadow]}
          activeOpacity={0.7}
          onPress={() => router.push(`/books/${book.id}`)}
        >
          <BookItemImg book={book} />
          <BookItemContent book={book} />
        </TouchableOpacity>
      ))}
    </View>
  );
};

const BookItemImg = ({ book }) => {
  return (

```

```

<View style={{ position: "relative" }}>
    <Image    source={book.img}    style={styles.book_card_img}
resizeMode="cover" />
    {!book.is_free && (
        <View style={[styles.circle_premium_small, styles.shadow]}>
            <AntDesign name="crown" size={18} color="black" />
        </View>
    )}
</View>

);
};

const BookItemContent = ({ book }) => {
    return (
        <View style={{ padding: 10 }}>
            <Text    style={styles.book_card_title}    numberOfLines={2}
ellipseMode="tail">
                {book.title}
            </Text>
            <Text style={styles.book_card_author} numberOfLines={1}>
                {book.author}
            </Text>
            <View style={styles.book_card_footer}>
                <View style={{ flexDirection: "row", alignItems: "center" }}>
                    <AntDesign name="star" size={14} color={color_list.orange} />
                    <Text style={styles.book_card_rating}>{book.rating}</Text>
                </View>
                {book.views && (
                    <View style={{ flexDirection: "row", alignItems: "center" }}>
                        <Icons name="eye-outline" size={14} color="gray" />
                        <Text style={styles.book_card_views}>{book.views}</Text>
                    </View>
                )}
            </View>
        </View>
    );
};

```

```
        </View>
      </View>
    );
  };
};
```

Source Code /components/Pagination.jsx

```
import { Text, TouchableOpacity, View } from "react-native";
import { color_list } from "../styles/StyleApps.js";

const MAX_VISIBLE_PAGES = 5;

export default function Pagination({ currentPage, totalPages, onPageChange })
{
  if (totalPages <= 1) {
    return (
      <View style={{ flexDirection: "row", justifyContent: "center", marginTop:
16 }}>
        <PageButton label="1" active onPress={() => {}} />
      </View>
    );
  }

  let start = Math.max(1, currentPage - Math.floor(MAX_VISIBLE_PAGES /
2));
  let end = start + MAX_VISIBLE_PAGES - 1;

  if (end > totalPages) {
    end = totalPages;
    start = Math.max(1, end - MAX_VISIBLE_PAGES + 1);
  }

  const pageNumbers = [];
  for (let i = start; i <= end; i++) pageNumbers.push(i);
```

```

const isFirstPage = currentPage === 1;
const isLastPage = currentPage === totalPages;

return (
  <View
    style={{
      flexDirection: "row",
      justifyContent: "center",
      alignItems: "center",
      marginTop: 16,
      gap: 6,
    }}
  >
    {!isFirstPage && (
      <>
        <PageButton label="<<" onPress={() => onPageChange(1)} />
        <PageButton label="<" onPress={() => onPageChange(currentPage - 1)} />
      </>
    )}

    {pageNumbers.map((page) => (
      <PageButton
        key={page}
        label={String(page)}
        active={page === currentPage}
        onPress={() => onPageChange(page)}
      />
    ))}

    {!isLastPage && (
      <>

```

```

        <PageButton label=">" onPress={() => onPageChange(currentPage + 1)}
      />
      <PageButton label=">>" onPress={() => onPageChange(totalPages)} />
    </>
  )}
</View>
);
}

```

```

const PageButton = ({ label, active = false, onPress }) => (

```

```

  <TouchableOpacity
    onPress={onPress}
    activeOpacity={0.7}
    style={{
      minWidth: 32,
      height: 32,
      paddingHorizontal: 8,
      borderRadius: 6,
      alignItems: "center",
      justifyContent: "center",
      backgroundColor: active ? color_list.green : "#fff",
      borderWidth: 1,
      borderColor: active ? color_list.green : "#ddd",
    }}

```

```

  >

```

```

  <Text
    style={{
      color: active ? "#fff" : "#333",
      fontWeight: active ? "700" : "400",
      fontSize: 13,
    }}
  >
    {label}

```

```
        </Text>
      </TouchableOpacity>
    );
```

Latihan 4: Laporan Singkat

1. Buatlah dokumentasi dalam mengerjakan soal Latihan 1 hingga Latihan 3, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project react expo yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB yang telah dibuat sebelumnya di repository "PPB-Praktikum-NPM", namun commit dengan nama "Tugas Praktikum-5".
3. Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).

LINK REPOSITORY

<https://github.com/RicoSteven120206/PPB-Praktikum-242310037.git>